

Cypress BDD + Cucumber/Gherkin Project Setup Guide

This project demonstrates how to set up a Cypress automation framework using Behavior-Driven Development (BDD) with Cucumber and Gherkin syntax.

Table of Contents

- [What is BDD?](#)
- [What is Cucumber?](#)
- [Why use Cucumber with Cypress?](#)
- [Gherkin keywords](#)
- [Required dependencies](#)
- [Project setup steps](#)
- [Project configuration](#)
- [Running the tests](#)
- [Folder structure](#)
- [Example feature file](#)
- [Import details](#)
- [Example step definitions](#)
- [Automation flow](#)
- [Setup checklist](#)

What is BDD?

BDD stands for Behavior-Driven Development.

BDD is a way of writing test cases in plain, readable language so that non-technical people such as Product Owners (POs), Project Managers (PMs), developers, and testers can understand the expected behavior of the application.

Key concepts

- BDD (Behavior-Driven Development)
- Cucumber
- Gherkin syntax
- Cypress
- Given
- When
- Then

In BDD, we describe the behavior of the application using keywords like **Given**, **When**, and **Then** and map them to automation code.

What is Cucumber?

Cucumber is a tool that supports BDD by allowing us to write test scenarios in Gherkin syntax and connect them with automation code.

Cucumber works with:

- Feature files
- Step definitions

Feature files

Feature files describe the behavior and functionality of the application using Gherkin syntax.

Step definitions

Step definitions contain the automation code mapped to the **Given**, **When**, **Then**, and other Gherkin steps.

Why use Cucumber with Cypress?

Using Cucumber with Cypress provides several benefits:

1. Collaboration
 - PMs, POs, developers, and QAs can read and understand test cases easily.
2. Readability
 - Tests are easier to understand because they are written in simple language.
3. Clear test documentation
 - Feature files act as documentation of expected behavior.
4. Goal achievement
 - The team can align better with product requirements and expected outcomes.
5. Reusability
 - Common steps can be reused across multiple scenarios.
6. BDD maintenance
 - Helps teams follow BDD practices in Agile and Scrum workflows.

Gherkin keywords

Gherkin provides several keywords used to describe application behavior.

1. Feature

A feature provides a high-level description of the functionality being tested.

2. Scenario

A scenario describes a specific behavior or test case with a few steps.

3. Given

Given describes the precondition or prerequisite of a test case. It defines the initial state before an action is performed.

4. When

When describes the action or event performed by the user, such as:

- Click
- Input
- Select
- Scroll
- Submit

5. Then

Then describes the expected result of the action. This is where assertions are typically performed.

6. And

And is used to add more steps to **Given**, **When**, or **Then** and makes scenarios easier to read.

7. Background

Background defines common steps shared by multiple scenarios within the same feature.

8. Scenario Outline

Scenario Outline is a template used to run the same scenario with multiple data rows from an **Examples** table.

Required dependencies

The project uses the following dependencies (see **devDependencies** in **package.json**):

Package	Version	Purpose
cypress	^16.0.0	End-to-end test framework
@badeball/cypress-cucumber-preprocessor	^28.0.0	Cucumber/Gherkin preprocessor for Cypress
@bahmutov/cypress-esbuild-preprocessor	^2.2.8	ESBuild bundler that compiles .feature files and step definitions
cypress-xpath	^2.0.1	Adds cy.xpath() support for XPath locators

Install them as dev dependencies:

```
npm install cypress --save-dev
npm install @badeball/cypress-cucumber-preprocessor --save-dev
npm install @bahmutov/cypress-esbuild-preprocessor --save-dev
npm install cypress-xpath --save-dev
```

⚠ Dependency change: the legacy **cypress-cucumber-preprocessor** package (from the older **TheBrainFamily** project) has been **replaced** with **@badeball/cypress-cucumber-preprocessor** (v28+). Its configuration keys also changed — see [Project configuration](#) below.

You also need:

1. Node.js 22.x (or newer)
 - Provides **npm** for installing packages.
 - This project's installed versions require: Cypress **^16.0.0** → Node **^22.0.0 || ^24.0.0 || >=26.0.0**, and **@badeball/cypress-cucumber-preprocessor** **^28.0.0** → Node **^20.12.0 || ^21.7.0 || >=22**.
2. npm
 - Package manager bundled with Node.js (npm 10.x ships with Node 22). Used to install the packages from **package.json**.
3. Visual Studio Code
 - Code editor used to write and manage the project.

Project setup steps

Step 1: Create and open the project folder

Create a new folder for the Cypress project and open it in VS Code.

```
code .
```

Step 2: Check Node.js and npm versions

```
node -v  
npm -v
```

Make sure both are installed and working properly before continuing.

Step 3: Initialize the Node.js project

```
npm init
```

This creates a **package.json** file.

Step 4: Install Cypress

```
npm install cypress --save-dev
```

or

```
npm i cypress --save-dev
```

This installs Cypress and adds it to **devDependencies** in **package.json**.

Step 5: Open Cypress

```
npx cypress open
```

This opens the Cypress app and may guide you through the initial setup.

Step 6: Install the Cucumber/Gherkin extension

Install the VS Code extension:

- Cucumber (Gherkin) language support

Link:

- <https://marketplace.visualstudio.com/items?itemName=alexkrechik.cucumberautocomplete>

This helps with:

- Gherkin syntax highlighting
- `.feature` file support
- Autocomplete
- Step-definition navigation

Step 7: Install the Cucumber preprocessor and supporting dependencies

This project uses `@badeball/cypress-cucumber-preprocessor` (the actively maintained package). The old `cypress-cucumber-preprocessor` package is deprecated and has been replaced.

```
npm install @badeball/cypress-cucumber-preprocessor --save-dev
npm install @bahmutov/cypress-esbuild-preprocessor --save-dev
npm install cypress-xpath --save-dev
```

Note: Always verify the installation instructions for the specific version you are using.

Step 8: Configure `cypress.config.js`

Use the configuration below — it matches the current project setup (`@badeball/cypress-cucumber-preprocessor` registered via `addCucumberPreprocessorPlugin` and compiled with the ESBuild bundler):

```
const { defineConfig } = require("cypress");

const {
  addCucumberPreprocessorPlugin,
} = require("@badeball/cypress-cucumber-preprocessor");

const createBundler = require("@bahmutov/cypress-esbuild-preprocessor");
```

```
const createEsbuildPlugin =
  require("@badeball/cypress-cucumber-preprocessor/esbuild");

module.exports = defineConfig({
  e2e: {
    specPattern: "cypress/e2e/features/*.feature",

    async setupNodeEvents(on, config) {
      // Register Cucumber
      await addCucumberPreprocessorPlugin(on, config);

      // Register ESBuild
      on(
        "file:preprocessor",
        createBundler({
          plugins: [createEsbuildPlugin.default(config)],
        })
      );

      return config;
    },
  },
});
```

This tells Cypress to:

- Look for `.feature` files under `cypress/e2e/features/` via `specPattern`.
- Register the Cucumber preprocessor in `setupNodeEvents`.
- Use the ESBuild bundler to compile step definitions and their imports.

Step 9: Configure `package.json`

Add the step definitions path using the `camelCase` key `stepDefinitions`:

```
"cypress-cucumber-preprocessor": {
  "stepDefinitions": "cypress/e2e/step_definitions/**/*.{js,mjs,ts,tsx}"
}
```

⚠ Do **not** use `step-definitions` (kebab-case) or `nonGlobalStepDefinitions` — they are silently ignored by `@badeball/cypress-cucumber-preprocessor` v28+, which causes `Step implementation missing for "..."` errors.

Ensure the JSON syntax remains valid.

Step 10: Create the Cucumber folder structure

```
cypress/
├─ Data/
└─ e2e/
```

```
├─ features/  
├─ Locators/  
├─ pages/  
└─ step_definitions/
```

Folder responsibilities

features/

Contains Cucumber feature files such as:

- `login.feature`
- `fund_request.feature`
- `customer_onboarding.feature`

pages/

Contains page object model files such as:

- `LoginPage.js`
- `DashboardPage.js`
- `FundRequestPage.js`

These files hold:

- Locators
- Page actions
- Reusable methods

step_definitions/

Contains step definition files such as:

- `login.js`
- `fund_request.js`
- `customer_onboarding.js`

These map the `.feature` file steps to Cypress automation code.

Locators/

Contains page locator classes (XPath/CSS selectors) used by the step definitions, e.g. `LoginLocator.js`.

Data/

Contains shared test data and constants (URLs, credentials, timeouts), e.g. `Shared_Data.js`.

Step 11: Configure the feature file pattern

In `cypress.config.js`:

```
specPattern: "cypress/e2e/features/*.feature"
```

This instructs Cypress to look in the `features` folder and execute `.feature` files.

Step 12: Configure the step definition path

In `package.json`:

```
"cypress-cucumber-preprocessor": {  
  "stepDefinitions": "cypress/e2e/step_definitions/**/*.{js,mjs,ts,tsx}"  
}
```

This glob connects feature files to the step definitions inside `cypress/e2e/step_definitions/`. All `.js`, `.mjs`, `.ts`, and `.tsx` files there (including subfolders) are picked up automatically.

Step 13: Project setup is complete

At this stage, the basic Cypress + Cucumber/Gherkin setup is ready.

Project configuration

The following is the **exact, current configuration** used in this project. Keep the format and the key names exactly as shown — `@badeball/cypress-cucumber-preprocessor` is strict about its configuration keys.

`cypress.config.js`

Full current file:

```
const { defineConfig } = require("cypress");  
  
const {  
  addCucumberPreprocessorPlugin,  
} = require("@badeball/cypress-cucumber-preprocessor");  
  
const createBundler = require("@bahmutov/cypress-esbuild-preprocessor");  
  
const createEsbuildPlugin =  
  require("@badeball/cypress-cucumber-preprocessor/esbuild");  
  
module.exports = defineConfig({  
  e2e: {  
    specPattern: "cypress/e2e/features/*.feature",  
  
    async setupNodeEvents(on, config) {  
      // Register Cucumber  
      await addCucumberPreprocessorPlugin(on, config);  
    },  
  },  
});
```



```

    // Register ESBUILD
    on(
      "file:preprocessor",
      createBundler({
        plugins: [createEsbuildPlugin.default(config)],
      })
    );

    return config;
  },
});

```

What each piece does:

- `specPattern: "cypress/e2e/features/*.feature"` — Cypress looks for `.feature` files only inside `cypress/e2e/features/`.
- `setupNodeEvents(on, config)` — runs in Cypress's Node context and registers everything the project needs:
 - `addCucumberPreprocessorPlugin(on, config)` — wires the Cucumber/Gherkin preprocessor into Cypress.
 - `createBundler({ plugins: [createEsbuildPlugin.default(config)] })` — the ESBUILD file preprocessor that compiles the step definitions and their imports (`cy.xpath`, `data/locator` classes, etc.).
- `return config;` — hands the (possibly modified) config object back to Cypress.

package.json (relevant part)

```

"devDependencies": {
  "@badeball/cypress-cucumber-preprocessor": "^28.0.0",
  "@bahmutov/cypress-esbuild-preprocessor": "^2.2.8",
  "cypress": "^16.0.0",
  "cypress-xpath": "^2.0.1"
},
"cypress-cucumber-preprocessor": {
  "stepDefinitions": "cypress/e2e/step_definitions/**/*.{js,mjs,ts,tsx}"
}

```

- `devDependencies` lists every package this project needs (see [Required dependencies](#) for their purpose).
- `"cypress-cucumber-preprocessor": { "stepDefinitions": ... }` — the glob the preprocessor uses to find step definitions. Subfolders of `cypress/e2e/step_definitions/` are included, and `.js`, `.mjs`, `.ts`, `.tsx` files are picked up automatically.

cypress/support/e2e.js

```
import './commands'  
import 'cypress-xpath'
```

- `import './commands'` — loads the default Cypress custom-commands file.
- `import 'cypress-xpath'` — registers `cy.xpath()`, the XPath command used by the step definitions.

⚠ Configuration changes to be aware of:

- The preprocessor key must be **stepDefinitions** (camelCase). Older guides showing **step-definitions** (kebab-case) or **nonGlobalStepDefinitions** are **outdated** — those keys are silently ignored. The preprocessor then falls back to its default search folders and you get **Step implementation missing for "..."**.
- **specPattern** must point to your feature files: `"cypress/e2e/features/*.feature"`.
- **nonGlobalStepDefinitions** and the old `on("file:preprocessor", cucumber())` style belong to the **legacy** `cypress-cucumber-preprocessor` package — they must **not** be used with `@badeball/cypress-cucumber-preprocessor v28+`.

Running the tests

Quick start (fresh clone)

```
git clone https://github.com/mohammadmunnamia0/Cypress-BDD-Framework-Automation.git  
cd Cypress-BDD-Framework-Automation  
npm install
```

`npm install` reads `package.json` / `package-lock.json` and installs all the **devDependencies** listed above. `node_modules/` is listed in `.gitignore`, so it is never committed — it must be installed on every fresh clone.

Interactive mode

```
npx cypress open
```

Opens the Cypress Test Runner. Select **E2E Testing**, pick a browser, then click **Login_Test.feature** to run the Login scenario.

Headless mode

```
npx cypress run
```

Runs every `.feature` file matched by `specPattern` and prints the results to the terminal — ideal for CI or a quick verification.

Run a single feature file

```
npx cypress run --spec "cypress/e2e/features/Login_Test.feature"
```

If the application under test is reachable, a successful run reports **1 scenario passed** (Login to the Optifin Application With Valid Credentials) with **3 steps passed** (Given → When → Then).

Troubleshooting

- **Step implementation missing for "..."** — the `cypress-cucumber-preprocessor` config key in `package.json` is probably wrong. Use the camelCase `stepDefinitions` key shown above; `step-definitions` / `nonGlobalStepDefinitions` are ignored by v28+.
- **cy.xpath not recognized / XPath locators do nothing** — make sure `import 'cypress-xpath'` is present in `cypress/support/e2e.js` and that `cypress-xpath` is in `devDependencies`.
- **Tests pass locally but not in CI** — make sure `npx cypress run` is used, the application under test is reachable from the CI machine, and Node.js meets the version requirements above.

Folder structure

```
├── cypress/
│   ├── Data/
│   │   └── Shared_Data.js
│   ├── e2e/
│   │   ├── features/
│   │   │   └── Login_Test.feature
│   │   ├── Locators/
│   │   │   └── LoginLocator.js
│   │   ├── pages/
│   │   └── step_definitions/
│   │       └── LoginTest.js
│   ├── fixtures/
│   │   └── example.json
│   └── support/
│       ├── commands.js
│       └── e2e.js
├── node_modules/
├── cypress.config.js
├── package.json
└── README.md
```

Example feature file

This project's feature file — `cypress/e2e/features/Login_Test.feature`:

Feature: Login Test

Login to the Application Optifin

Scenario: Login to the Optifin Application With Valid Credentials

Given User is on the Login Page

When User enters valid username and password

Then User Click Login Button

Import details

Pay close attention to the imports — use the **exact format** used in the project files.

Step definition files (`cypress/e2e/step_definitions/LoginTest.js`):

```
import {Given, When, Then} from "@badeball/cypress-cucumber-preprocessor";
import Alldata from "../../Data/Shared_Data";
import User_Signup_locatorsPage from "../Locators/LoginLocator";
```

- `Given`, `When`, `Then` are **named imports** from `@badeball/cypress-cucumber-preprocessor` — no `.default` is needed. `And`, `But`, `Before`, `After`, etc. can be added to the same import block.
- `Shared_Data.js` and the locator classes are **default imports** of a class and are instantiated once per file:

```
const alldata = new Alldata();
const locators = new User_Signup_locatorsPage();
```

- Relative import paths are resolved **from the importing file's own folder**:
 - `../../Data/Shared_Data` → from `cypress/e2e/step_definitions/` up two levels → `cypress/Data/Shared_Data`
 - `../Locators/LoginLocator` → from `cypress/e2e/step_definitions/` up one level → `cypress/e2e/Locators/LoginLocator`
- `cypress.config.js` uses Node-style `require(...)` (it runs in Cypress's Node context), while step definition files use ES `import` statements.

Example step definitions

File: `cypress/e2e/step_definitions/LoginTest.js`

```
import {Given, When, Then} from "@badeball/cypress-cucumber-preprocessor";
import Alldata from "../../Data/Shared_Data";
import User_Signup_locatorsPage from "../Locators/LoginLocator";
```

```
const alldata = new Alldata();
const locators = new User_Signup_locatorsPage();

Given('User is on the Login Page', () => {
  cy.visit(alldata.Prime_SCF_url);
});

When('User enters valid username and password', () => {
  cy.xpath(locators.UserID).type('cad_duo');
  cy.xpath(locators.Password).type('Prime123@');
  cy.xpath(locators.Login).click();
});

Then('User Click Login Button', () => {
  cy.xpath(locators.Login).click();
});
```

Notes:

- The step text inside `Given('...')`, `When('...')`, `Then('...')` must match the Gherkin step text in the `.feature` file.
- `cy.xpath()` comes from the `cypress-xpath` package, registered once in `cypress/support/e2e.js` with `import 'cypress-xpath'`.
- `alldata.Prime_SCF_url` is the application URL stored in `cypress/Data/Shared_Data.js`.

Automation flow

```
Feature File
  ↓
Gherkin Scenario
  ↓
Step Definitions
  ↓
Page Objects
  ↓
Cypress Commands
  ↓
Application Under Test
```

This structure keeps the framework organized, reusable, maintainable, and easy to understand.

Setup checklist

Before starting test-case development, make sure you have completed the following:

- ☐ Node.js 22+ installed
- ☐ npm installed
- ☐ Node.js project initialized

- ☐ Dependencies installed (`npm install`)
- ☐ Cypress installed
- ☐ Cypress successfully opened
- ☐ Cucumber/Gherkin extension installed
- ☐ Cucumber preprocessor installed
- ☐ `cypress.config.js` configured
- ☐ `package.json` configured
- ☐ `cypress/support/e2e.js` updated (`import 'cypress-xpath'`)
- ☐ `features` folder created
- ☐ `Locators` folder created
- ☐ `Data` folder created
- ☐ `pages` folder created
- ☐ `step_definitions` folder created
- ☐ Feature file pattern configured (`specPattern` in `cypress.config.js`)
- ☐ `stepDefinitions` path configured (`camelCase` key in `package.json`)
- ☐ `cypress-xpath` registered in `cypress/support/e2e.js`
- ☐ Tests run successfully (`npx cypress run` / `npx cypress open`)

Next step

Project setup is complete. The next step is to write your first feature file, create the corresponding step definitions, and implement the page object model with Cypress.

This README was formatted for GitHub and project readability.